

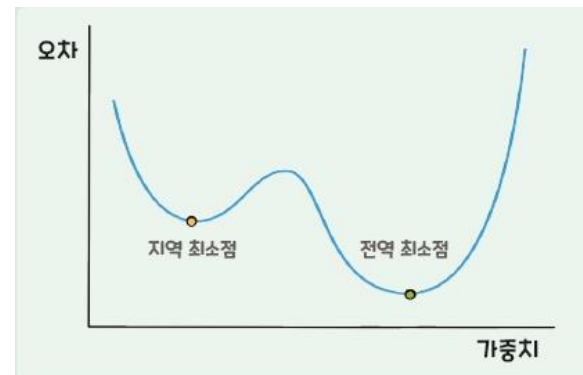
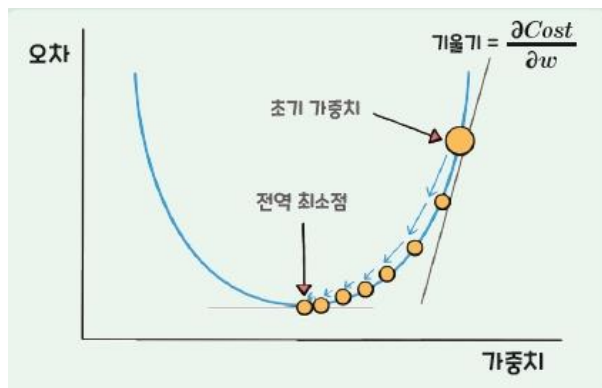
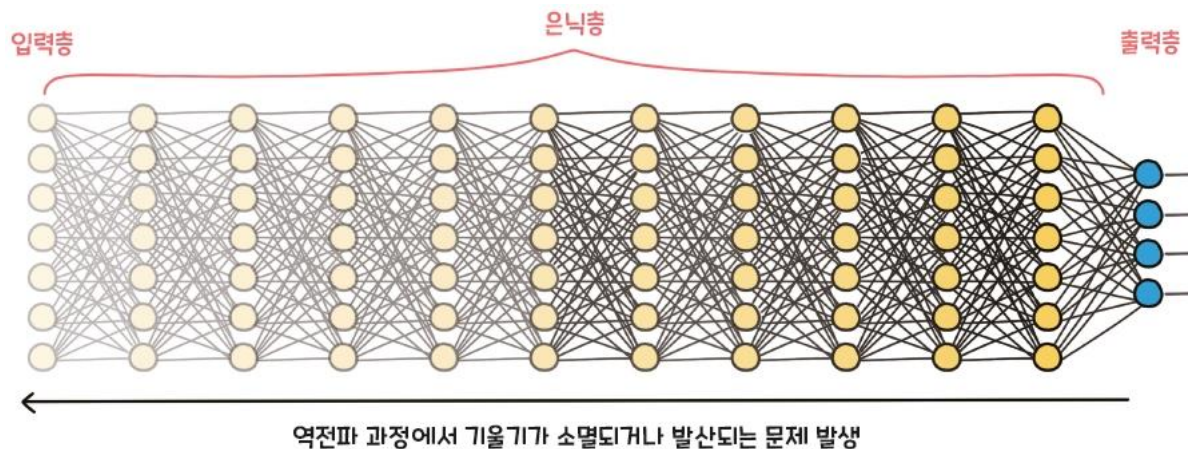
Chap.6 딥러닝 & 강화학습 기초

방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

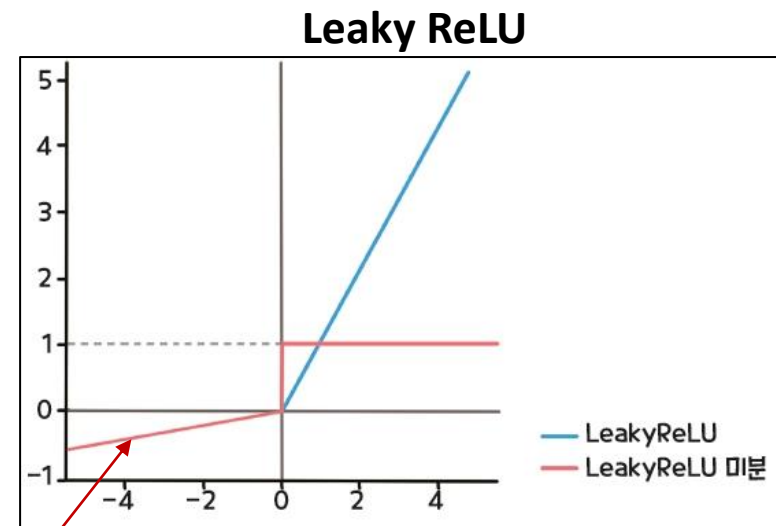
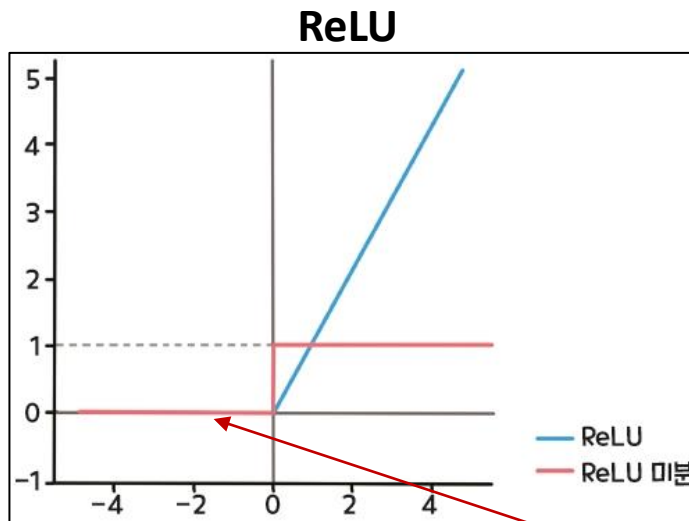
2024년도 1학기
머신러닝실습 & 인공지능설계실습1

- 신경망에서의 Back-propagation 문제
 - Weight Update를 위한 Cost Function의 기울기 값이 소멸되는 문제 발생
 - ReLU 함수의 발견으로 문제 해결 => 딥러닝의 시대 도래



• ReLU 함수

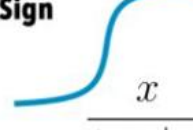
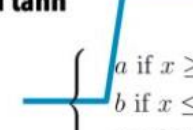
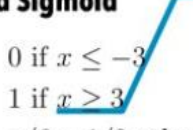
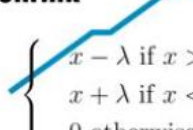
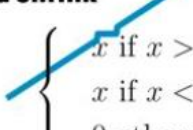
- x 가 음의 값을 가지면 0을 출력하고, 양의 값을 가지면 x 를 그대로 출력
- 함수 형태도 $\max(0, x)$ 로 계산이 간단하여 학습 속도가 매우 빠름



Node가 완전히 꺼져버리는 “Dead Neuron” or “Dying ReLU” 현상 보완

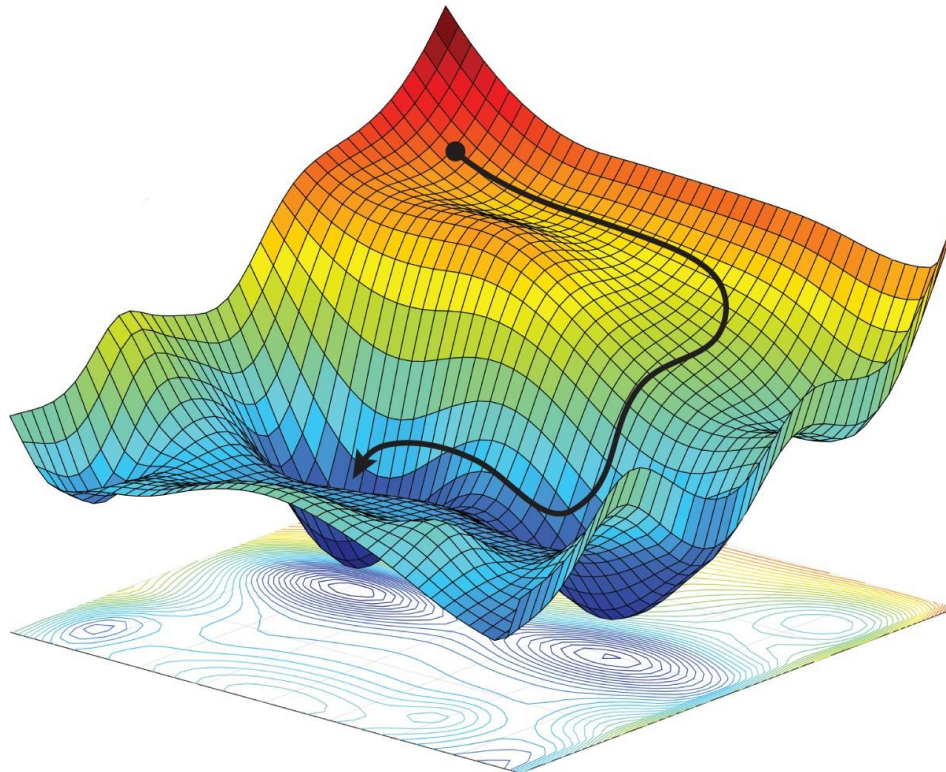
다양한 Activation Function



ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

- 옵티마이저

- 비용함수를 최소화 시키는 최적의 모델 파라미터를 찾아주는 알고리즘

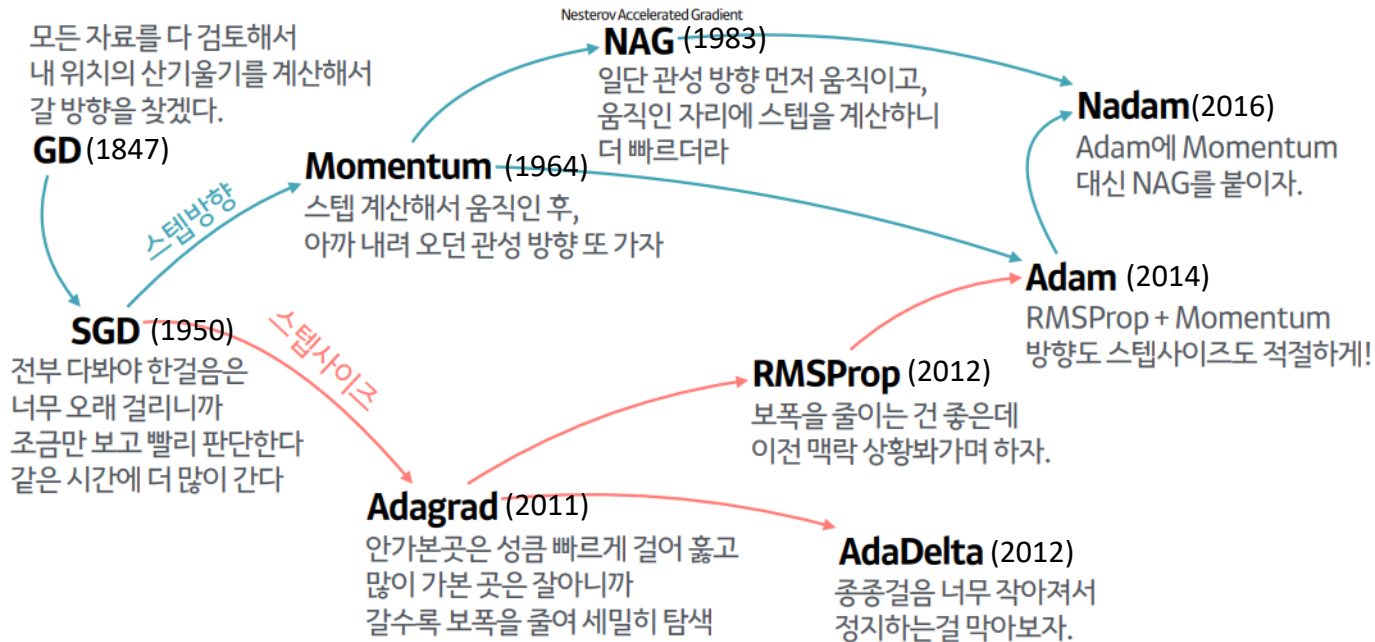


출처 : https://codetorial.net/tensorflow/basics_of_optimizer.html

■ 다양한 옵티마이저

- 최적의 모델 파라미터로 가는 경로는 다양하기 때문에
옵티마이저도 여러 알고리즘으로 개발

산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보

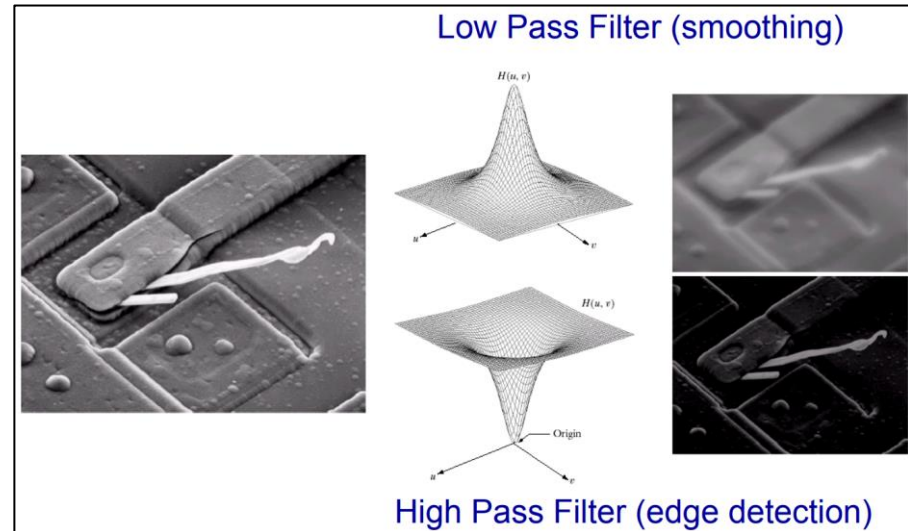
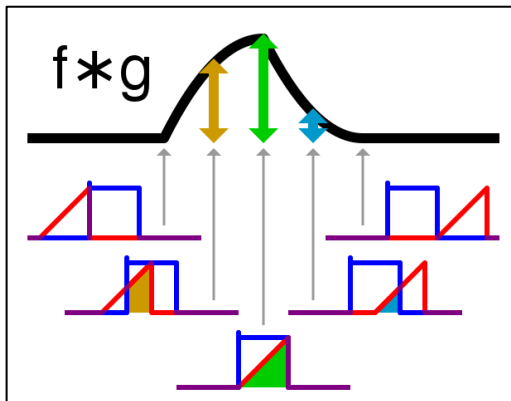
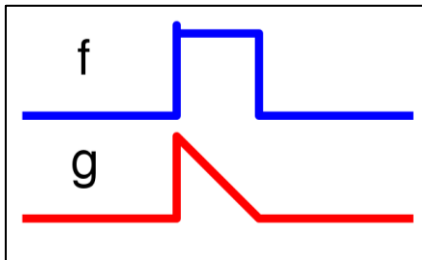


■ Convolutional Neural Network (CNN)

- 대표적인 이미지 분류 모델

- Convolution Filter를 활용하여 이미지의 특징을 두드러지게 함.

❖ Convolution의 정의: $(f * g)(t) \stackrel{\text{def}}{=} \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$



Convolutional Neural Network (CNN)

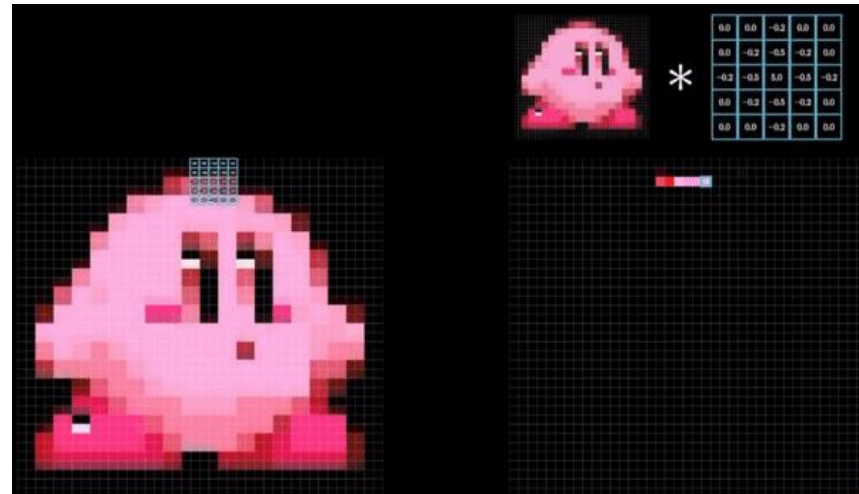


지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

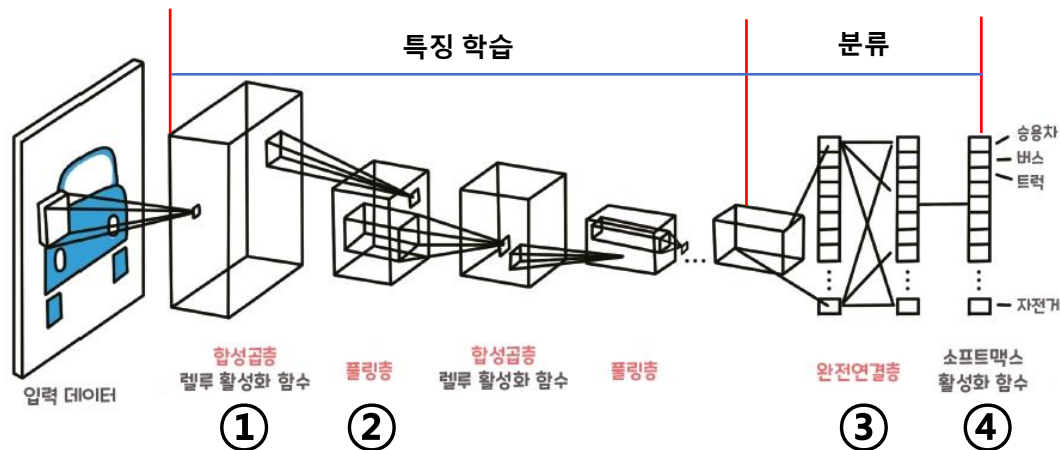
■ CNN (합성곱 신경망)

- 딥러닝의 가장 대표적인 방법, 주로 이미지 인식에 많이 사용됨
- 합성곱 신경망의 구조

- ① 합성곱층
- ② 풀링층
- ③ 완전연결층
- ④ 활성화함수

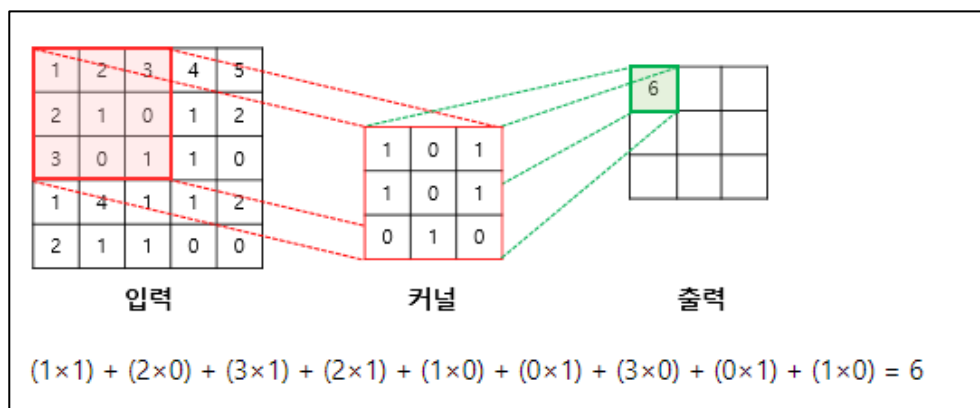


Convolution을 이용한 특징 추출



Convolution Layer

- Kernel (혹은 Filter)로 불리는 행렬을 활용하여 Convolution 연산



Kernel

1	0	1
0	1	0
1	0	1



1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

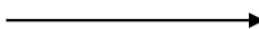
고려할 수 있는 Option은 무엇이 있을까?

■ Convolution Layer의 Option

- **Kernel의 수**: n by m 의 행렬에서 n 과 m 을 결정
- **Kernel의 Size**: 하나의 이미지에 대해서 몇 개의 필터를 활용할 것인가?
- **Stride**: Kernel을 몇 칸씩 움직이며 필터링할 것인가?
- **Zero Padding**: 이미지 전체의 사이즈를 유지 (외곽의 이미지의 특징도 활용하기 위해)

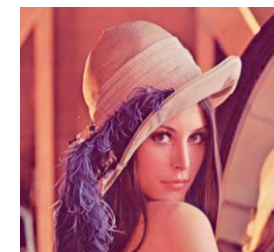
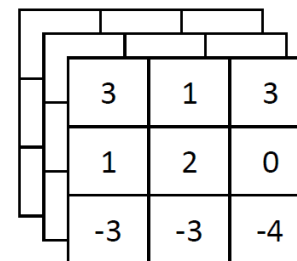
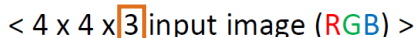
1	3	2	4
2	5	6	7
2	1	2	4
3	2	1	0

Zero Padding = 1



0	0	0	0	0	0
0	1	3	2	4	0
0	2	5	6	7	0
0	2	1	2	4	0
0	3	2	1	0	0
0	0	0	0	0	0

3 Channel



3	1	3
1	2	0
-3	-3	-4

+

2	2	1
3	0	1
-2	1	-1

+

3	1	1
2	2	2
-3	-2	-1

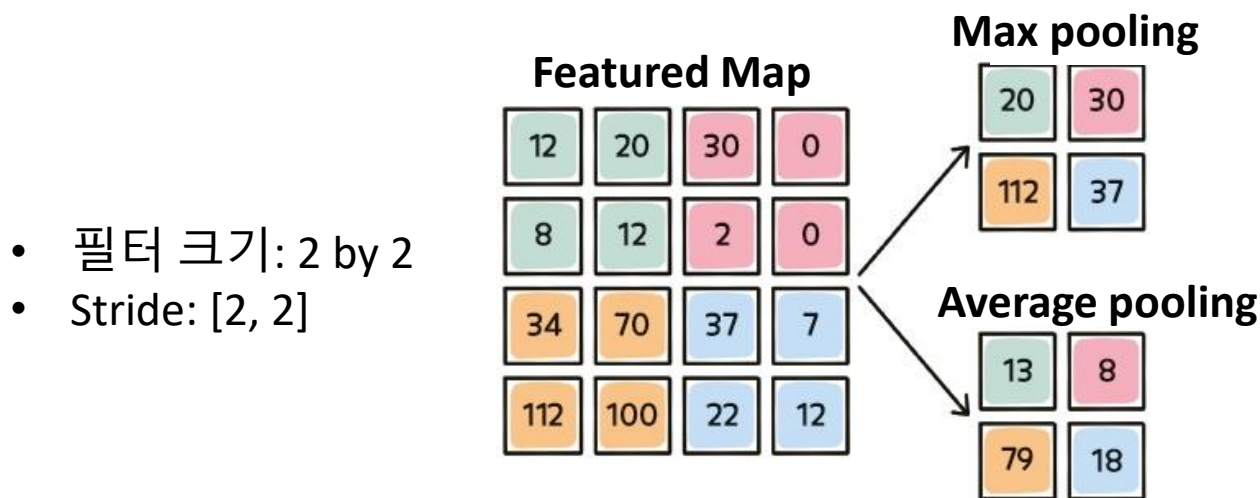
+Bias
(3x3)

8	4	5
6	4	3
-8	-4	-6

10

■ Pooling Layer

- Convolution Layer의 출력(Featured Map)을 입력으로 받아서 데이터의 크기를 줄이는데 활용
- Convolution Layer 마다 이미지 연산을 반복 수행할 경우, 연산량이 급격하게 증가하는 것을 방지
- 필터 종류, 크기, Stride를 결정해주어야 함.

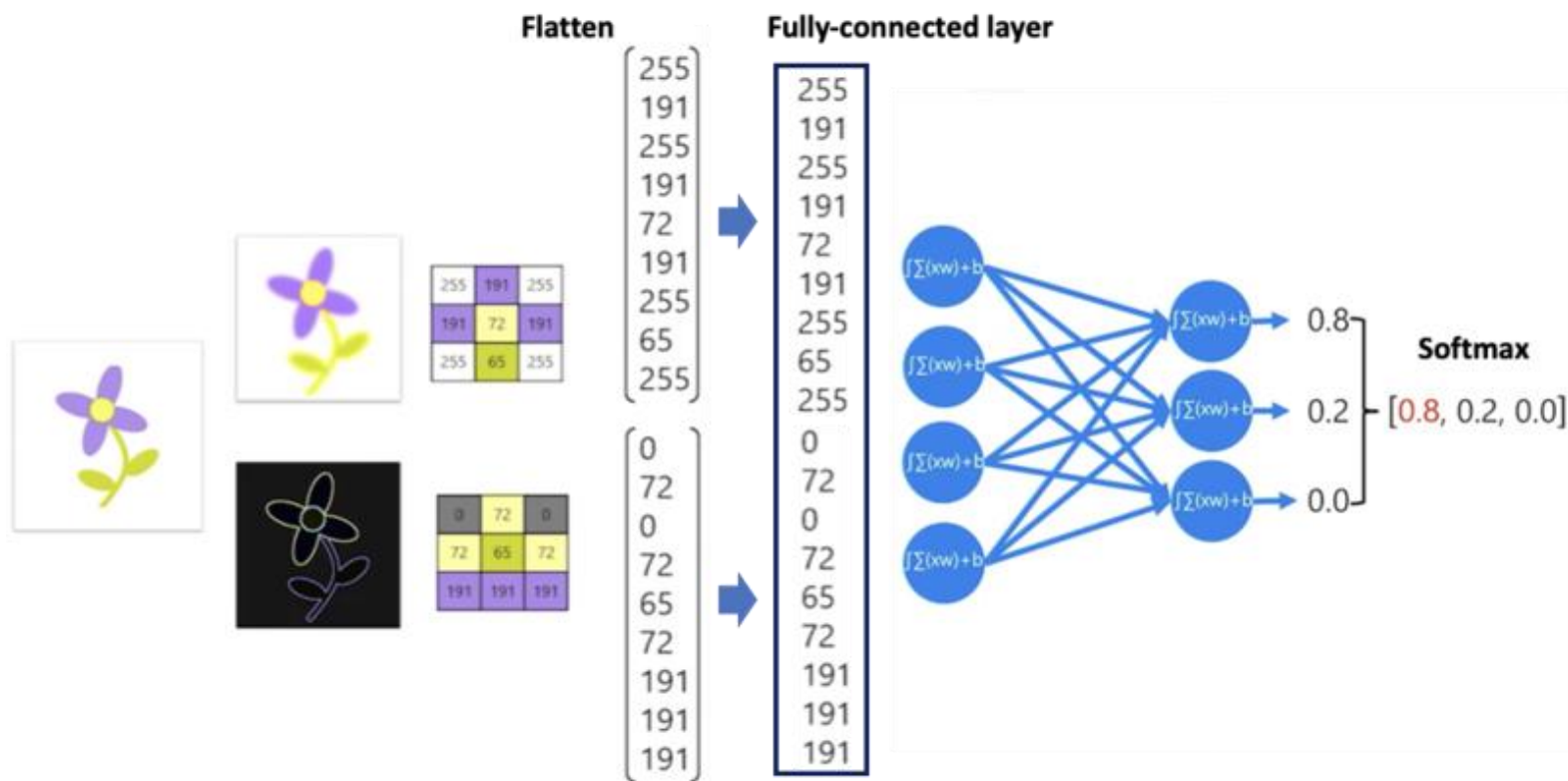


CNN (합성곱 신경망)



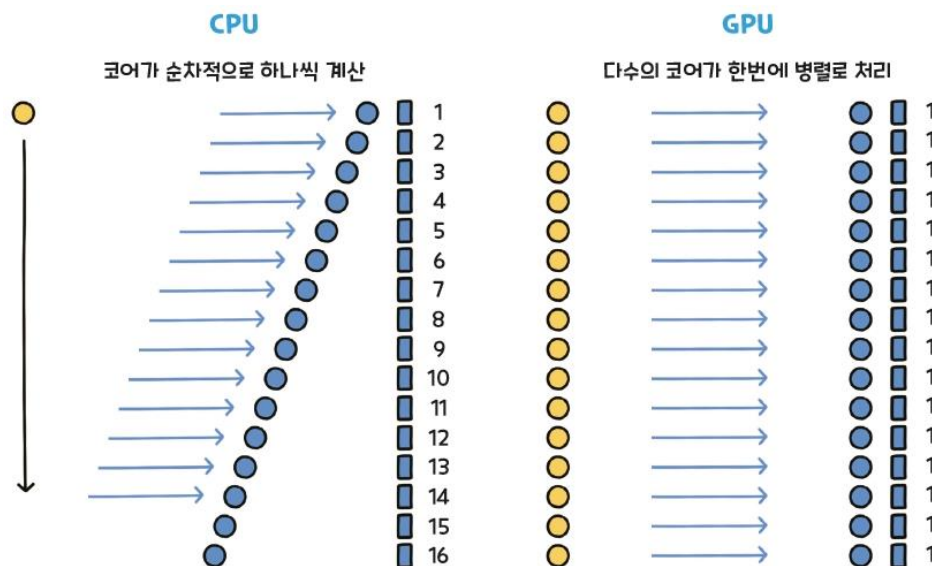
■ Fully-Connected Layer

- 최종 도출된 Featured Map을 하나의 신경망에 연결
- FC Layer의 출력 층 활성화 함수: Softmax



■ GPU 활용 검토

- 이미지, 영상과 같이 Massive Data에 대해 연산하기 위해서는 병렬 연산이 고려되어야 함.
- Training 과정에서 각 코어가 개별적으로 연산한 데이터를 중간 과정에서 수시로 평균을 취해 합치는 과정(동기화)을 거침.
- CPU 연산만으로도 충분한 성능을 낼 경우, GPU 기반 병렬 연산은 지양

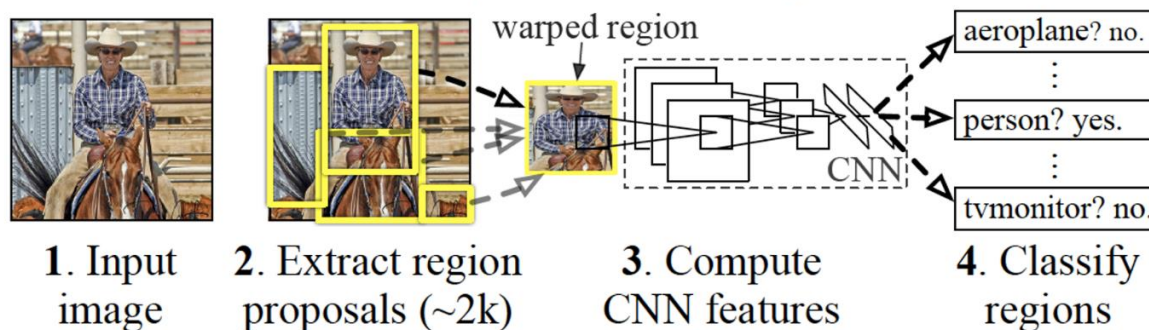


■ 이미지 내 객체 인식을 위한 모델

• R-CNN

- 이미지의 영역을 결정하는 Region Proposal 층이 추가됨.
- 이미지에 대해서 Sweeping, 즉 여러 번 검색하는 과정을 거침
- 많은 객체를 검출하는데 한계 존재

R-CNN: *Regions with CNN features*



Task: →

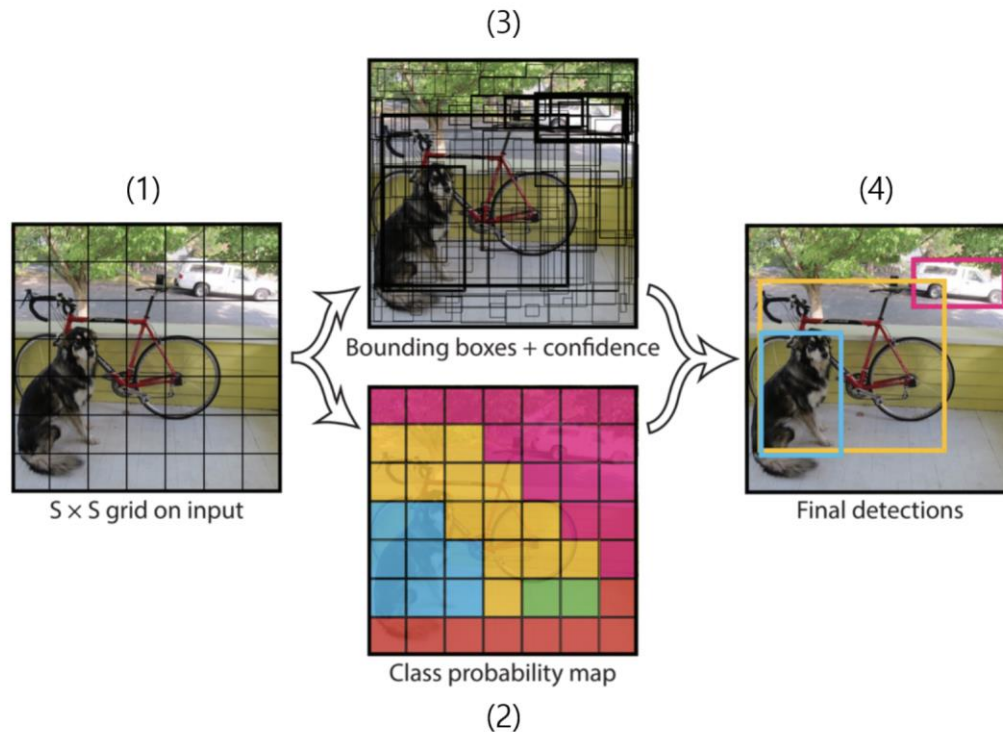
1. Region Proposal

2. Region Classification

■ 이미지 내 객체 인식을 위한 모델

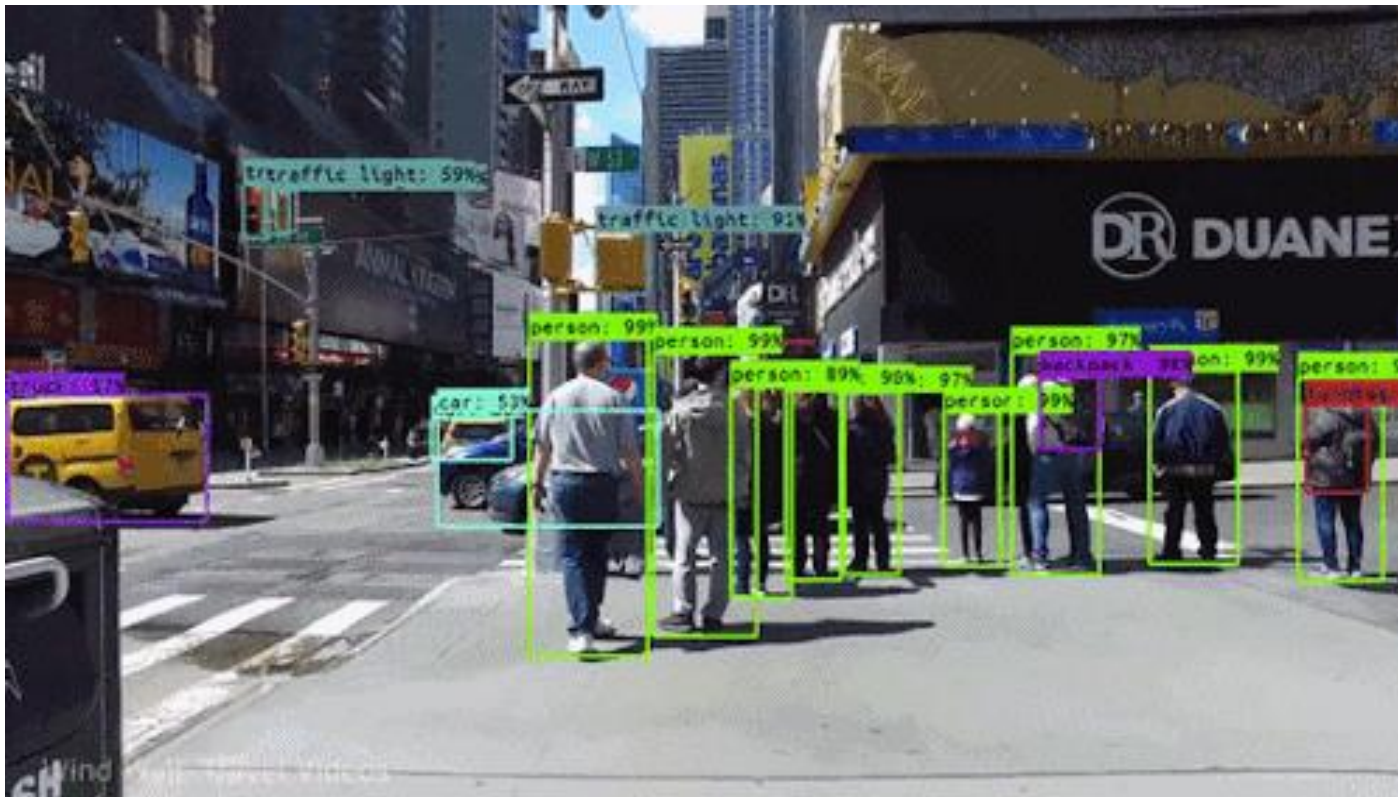
- YOLO (You Only Look Once)

- Region Proposal 층이 삭제되면서, 네트워크 하나로 지역 구분과 분류를 한번에 결정함.



■ YOLO (You Only Look Once)

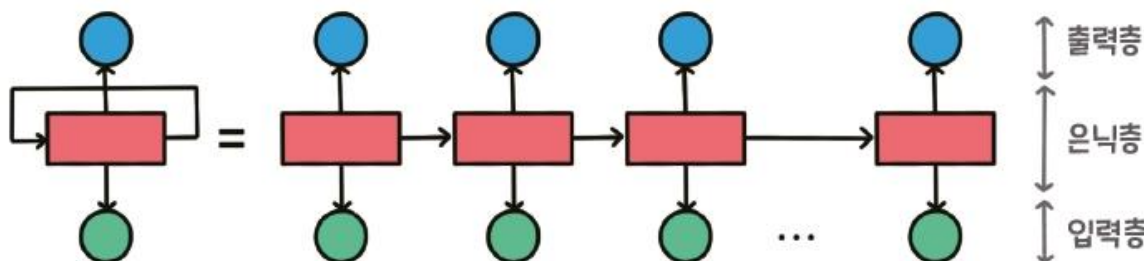
- “한번만 보겠다” 속도가 빠르다는 장점을 내세운 Object Detection 모델
- **Localization**과 **Classification**을 동시에 진행되도록 설계
- 대표적인 **1-stage Detector**의 모델 중 하나



■ Recurrent Neural Network (RNN)

- 데이터 간의 연결성을 보존하는 신경망

- 시계열 데이터 (음성 신호 등)
- 문자열

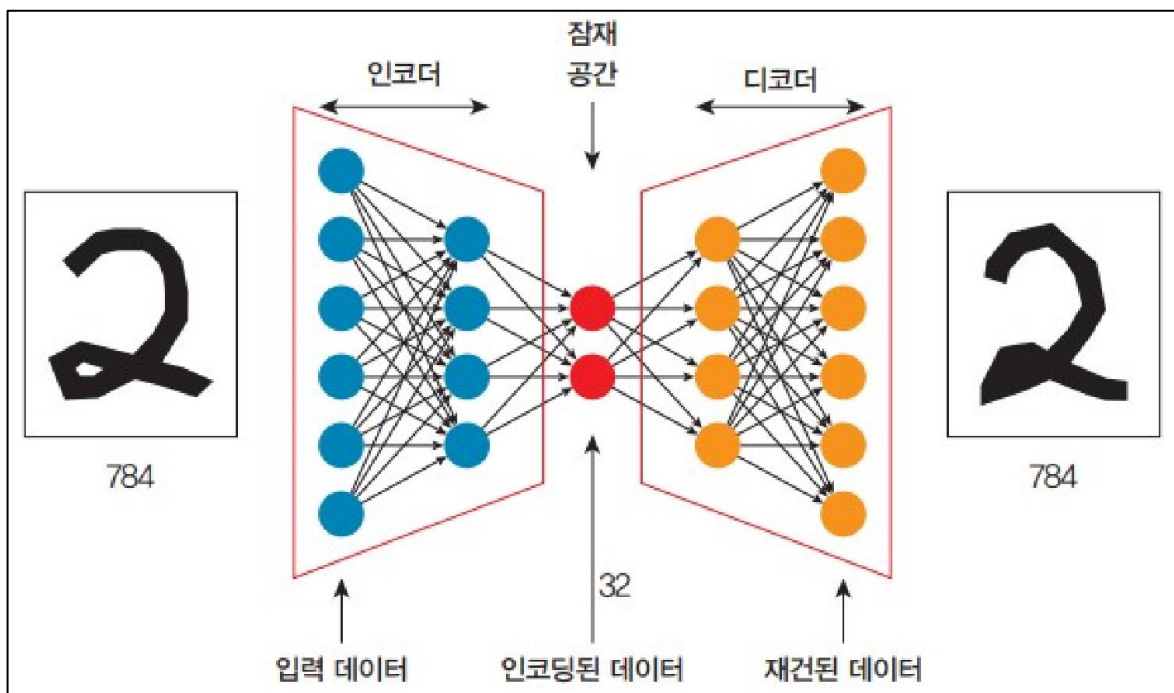


- 대표적 RNN: Long Short-Term Memory (LSTM)

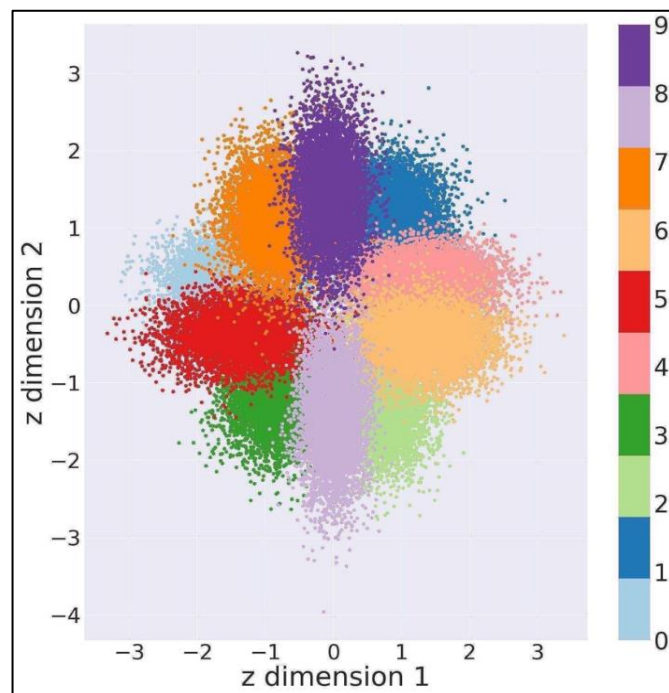
- Deep할수록 먼 과거의 데이터가 현재에 영향을 미치지 못하는 문제 (Long-Term Dependency) 를 해결하기 위해 고안

■ 오토인코더 (Auto-Encoder, AE)

- 고차원 데이터를 저차원 벡터로 축소하는 Encoder와 축소된 벡터를 원래 데이터로 복원하는 Decoder를 신경망으로 자동 설계



AE의 구조



Latent space

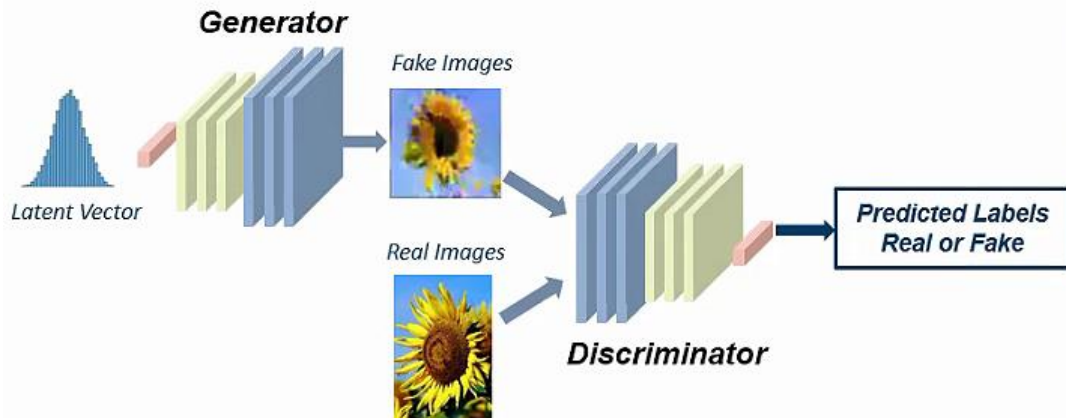
■ Generative Adversarial Network(GAN)

- 비지도학습의 선두 주자

- Generator: 노이즈로부터 Fake image를 생성하는 신경망
- Discriminator: Input image가 Real인지 Fake인지 분류하는 이진 분류 신경망
- 두 신경망이 적대적(경쟁적)으로 학습되는 구조

=> 매우 사실적 이미지를 생성할 수 있음.

Train to trick the Discriminator



Train to judge real / fake correctly



Generate an image similar to real images

생산적 적대 신경망(GAN)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

- '콜로라도 주립 박람회 미술대회' 디지털아트 1위
'갑론을박'..."로봇이 올림픽 참가한 꼴" VS "도구일 뿐"



스페이스 오페라 극장

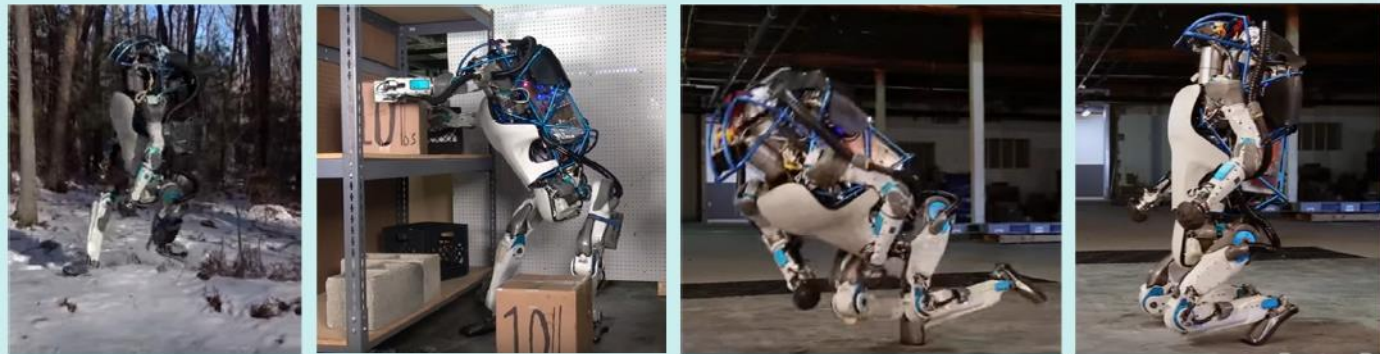
디지털아트 부문에서 게임 기획자 제이슨 M.앨런(39)

■ 강화 학습의 과업 사례

- [그림 9-1]의 과업은 상태 변화와 행동을 번갈아 수행하면서 목적을 달성
- 지도 학습이나 비지도 학습으로는 과업 달성 불가능 → 강화 학습으로 해결



(a) 사람의 걷기 학습 과정



(b) 2족 로봇의 걷기

그림 9-1 걷기 과업

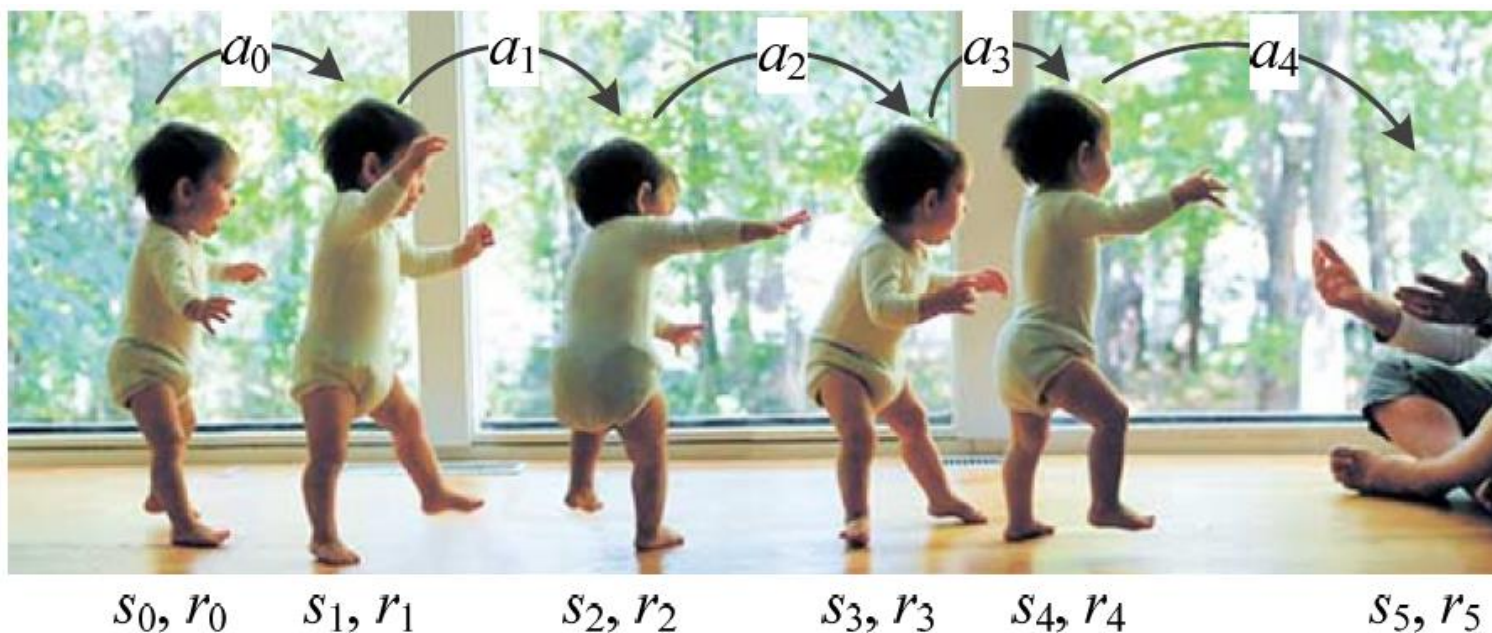
■ 강화 학습의 핵심 연산

- 시간에 따른 상태state, 행동action, 보상reward의 순차적 처리 $t=0, 1, 2, 3, \dots, T$

- $s_0, r_0 \xrightarrow{a_0} s_1, r_1 \xrightarrow{a_1} s_2, r_2 \xrightarrow{a_2} \dots \xrightarrow{a_{T-1}} s_T, r_T$

- 식으로 쓰면,

$$\text{강화 학습의 핵심 연산 } f: (s_t, a_t) \rightarrow (s_{t+1}, r_{t+1}) \quad (9.1)$$



[그림 9-2] 강화 학습의 핵심 개념인 상태 s_i , 행동 a_i , 보상 r_i

■ 에이전트와 agent 환경 environment

- 에이전트는 정책을 가지고 행동을 결정(정책은 학습으로 알아내야 함)
- 환경은 MDP를 가지고 상태 전환과 보상액 결정(MDP는 주어지는 것: 지도 학습의 훈련집합에 비유할 수 있음)

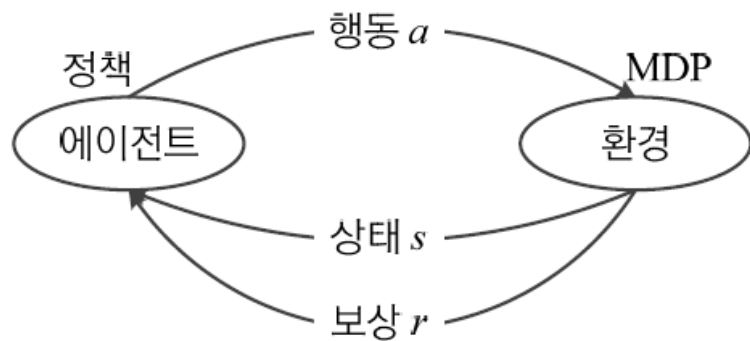


그림 9-3 강화 학습의 계산 모델

■ 강화 학습의 목표는 누적 보상액을 최대화하는 것

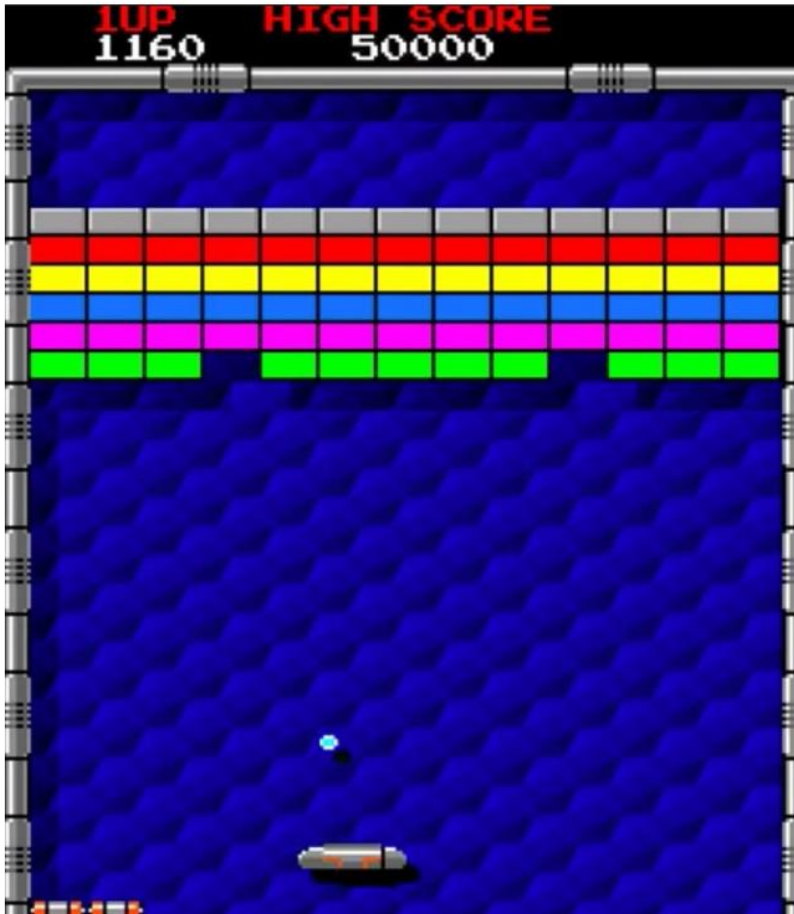
- 이를 달성하려면 매 순간 좋은 행동을 취할 수 있어야 함 → 정책에 따라 행동을 결정하므로 좋은 정책이 필요한 셈
- 강화 학습은 주어진 MDP를 가지고 최적 정책을 찾아야 함(지도 학습이 훈련집합을 가지고 최적 매개변수를 찾는 것에 비유할 수 있음)

Breakout (벽돌깨기 게임)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

조금 단순하게 접근해 봅시다.



게임의 목표는?

해당 그림만 보았을 때, 최적의 Action은?

고득점을 얻기 위한 전략은?

본 게임에서 취할 수 있는 Action?

Breakout 강화학습 적용



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.



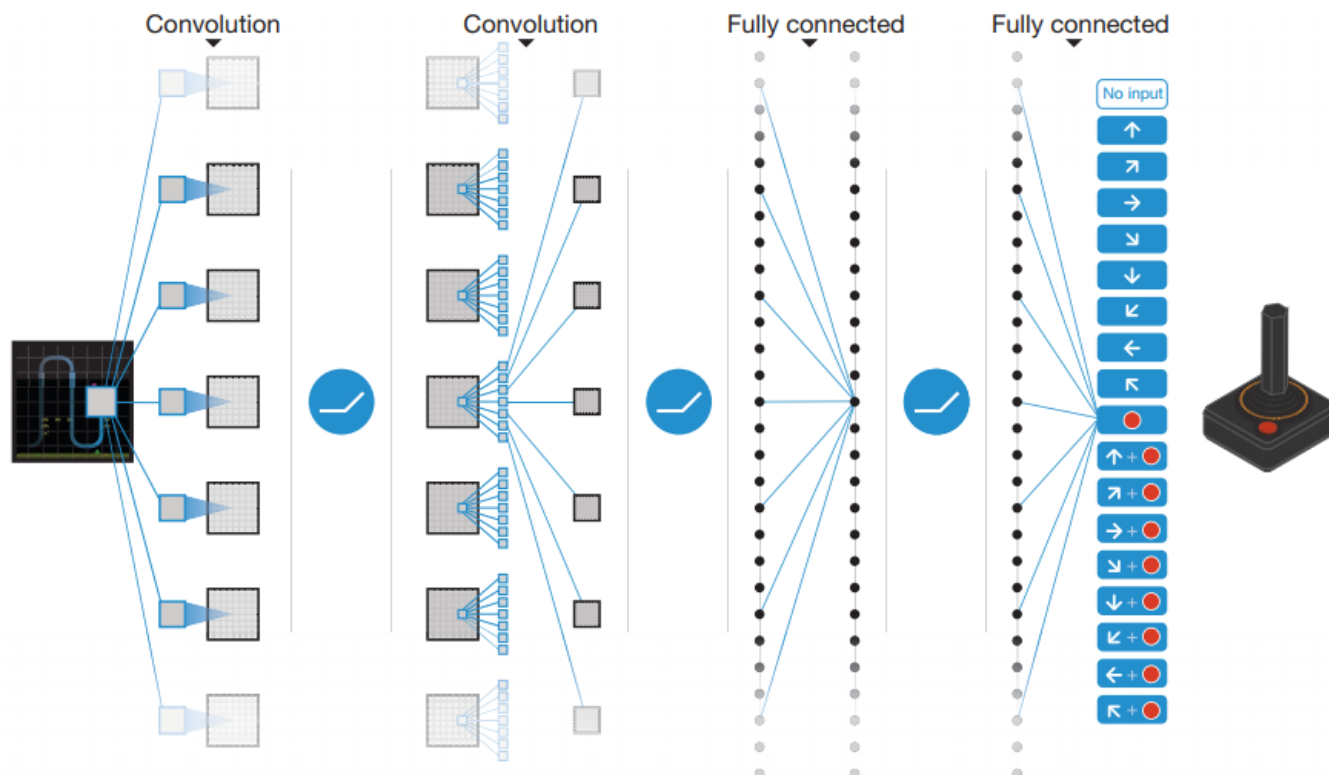
<https://www.youtube.com/watch?v=TmPfTpjtdgg>

Breakout 강화학습 적용



■ 딥러닝 + 강화학습 알고리즘 결합

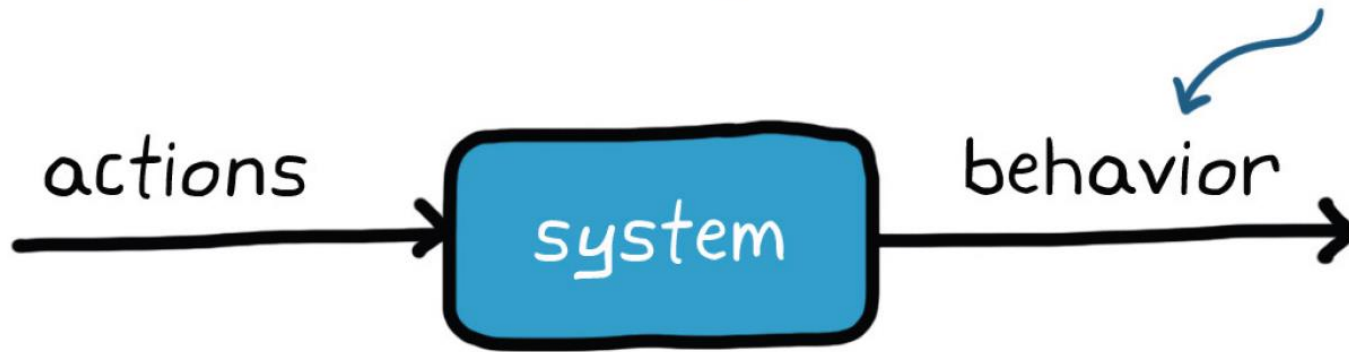
- 컨볼루션 신경망(CNN, 딥러닝의 일종)과 Q-학습(강화학습 알고리즘)을 결합한 모델
- 입력층은 게임 스크린 영상(최근 4장의 영상 $84 \times 84 \times 4$ 텐서) ← 이 텐서가 상태를 표현
- 출력층은 18개 노드(게임마다 4~18개 조이스틱 행동이 있는데 게임마다 필요한 만큼 사용)



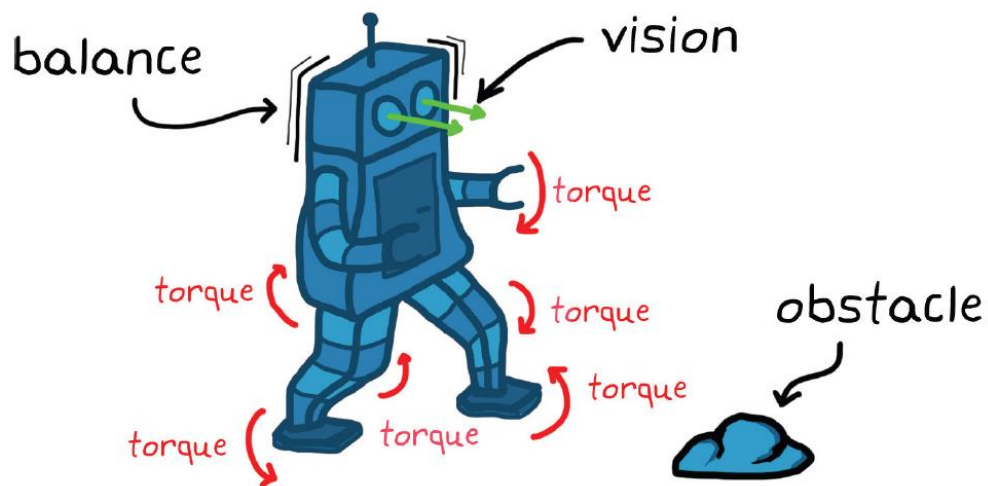
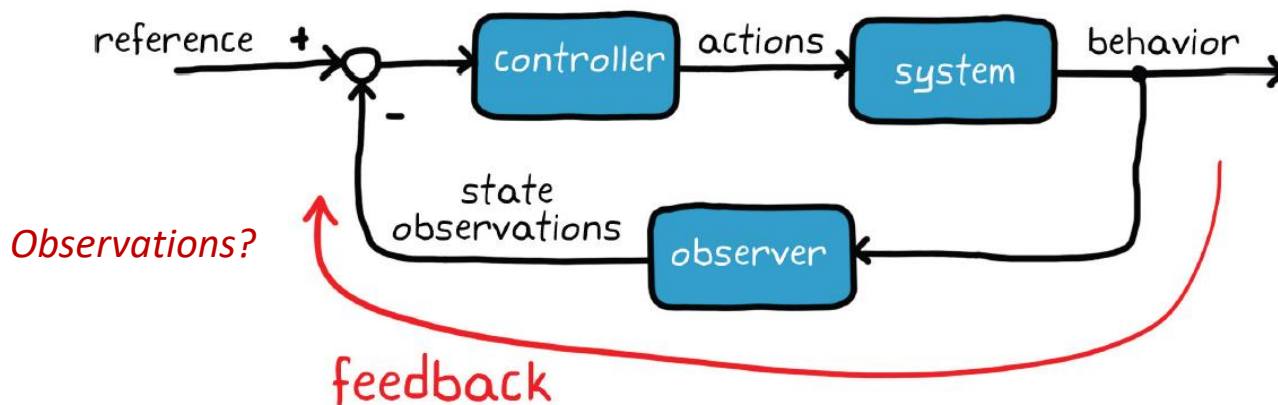
Goal to Control



which actions generate the desired behavior?



이쪽 보행 로봇에서의 Actions?



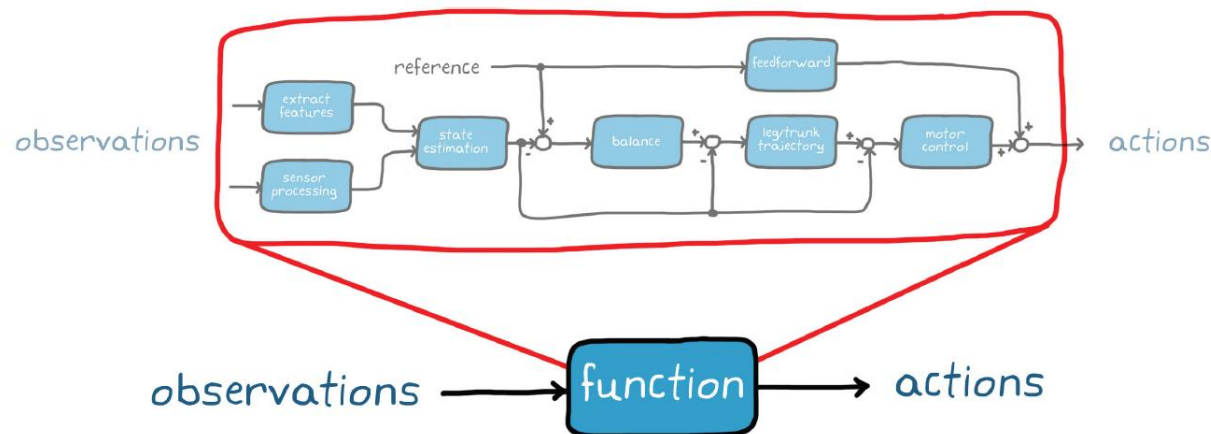
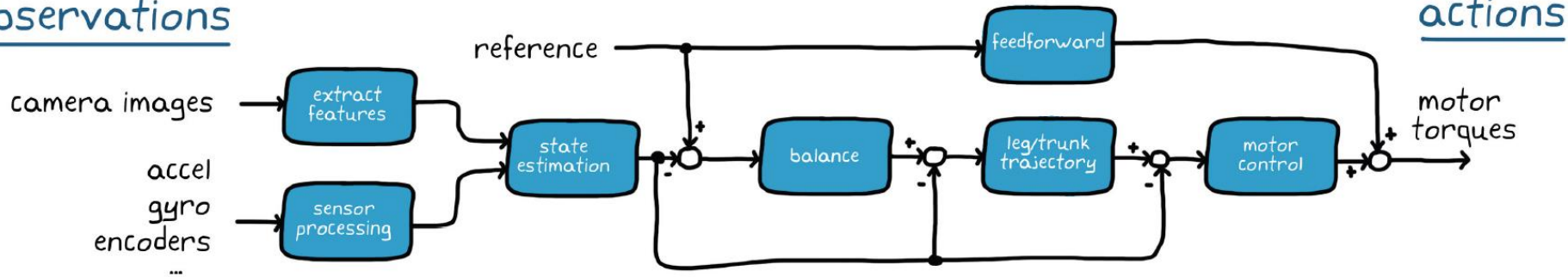
어떻게 Controller가 Action 값을 조절할 것인가?

제어공학 vs 강화학습



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

observations



what does this function look like?

how do you design it?

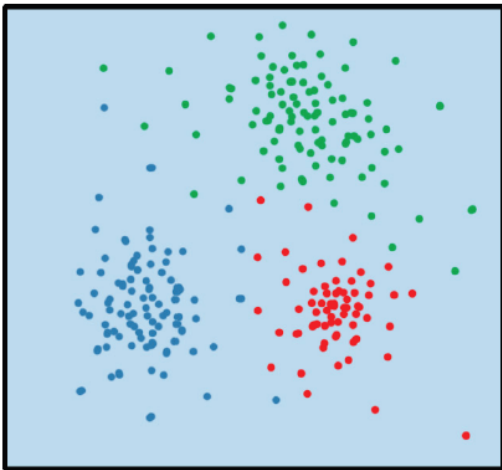
Reinforcement Learning!

machine learning

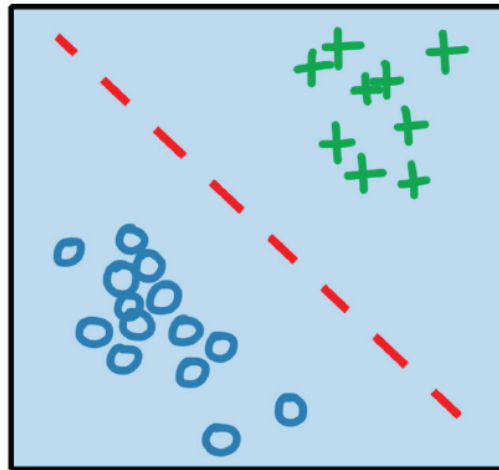
unsupervised
learning

supervised
learning

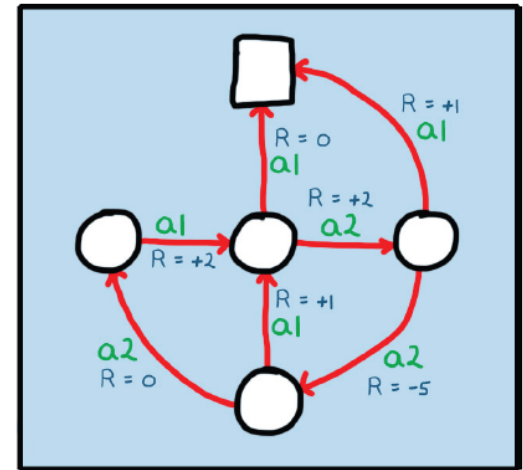
reinforcement
learning



목표: Labeling



목표: Classification



목표??

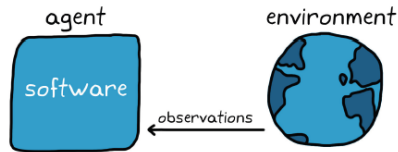
강화학습 목표와 Training 프로세스



- 목표: Find the best actions that will generate the optimal outcome

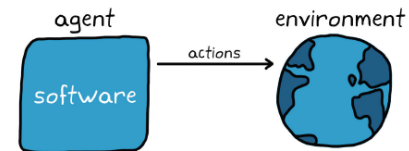
1

The agent is able to observe the current state of the environment.



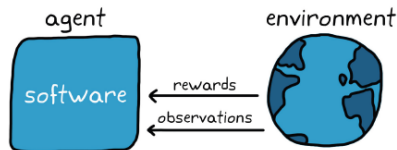
2

From the observed state, it decides which action to take.



3

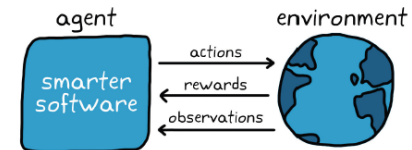
The environment changes state and produces a reward for that action. Both of which are received by the agent.



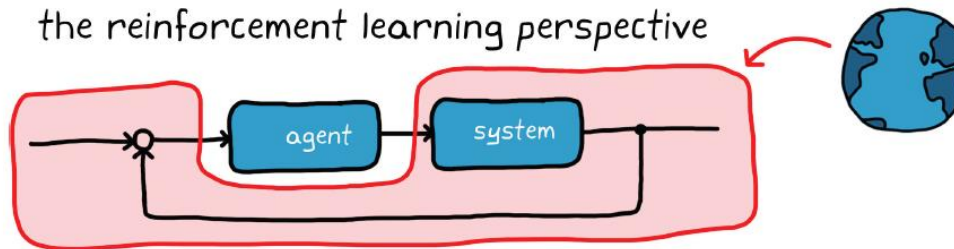
4

Using this new information, the agent can determine whether that action was good and should be repeated, or if it was bad and should be avoided.

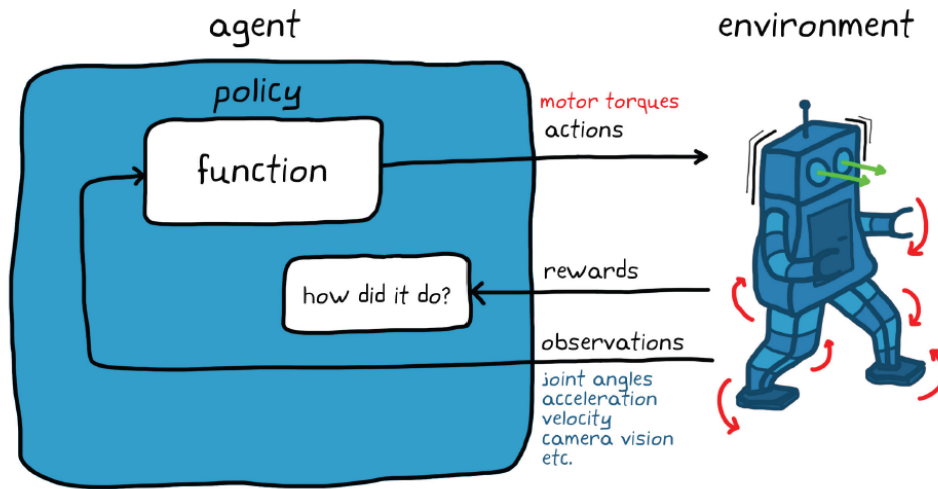
The observation-action-reward cycle continues until learning is complete.



the reinforcement learning perspective



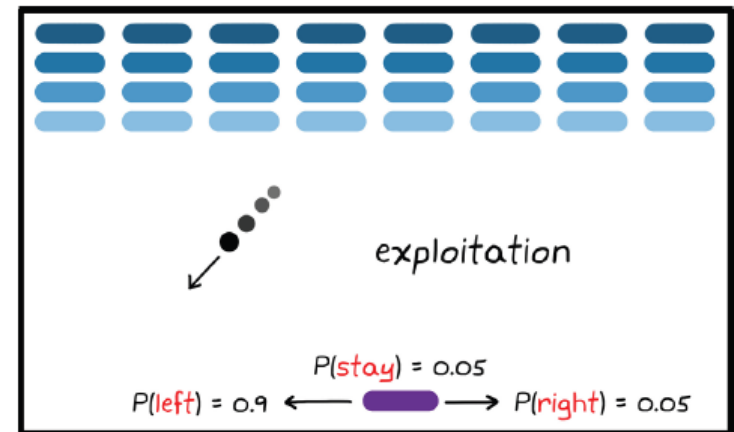
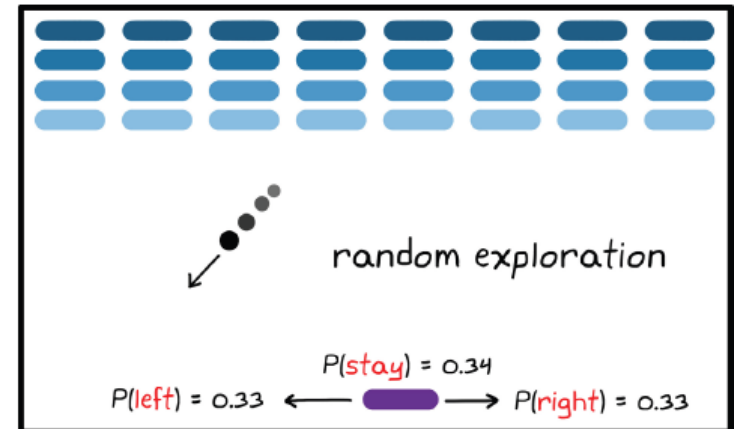
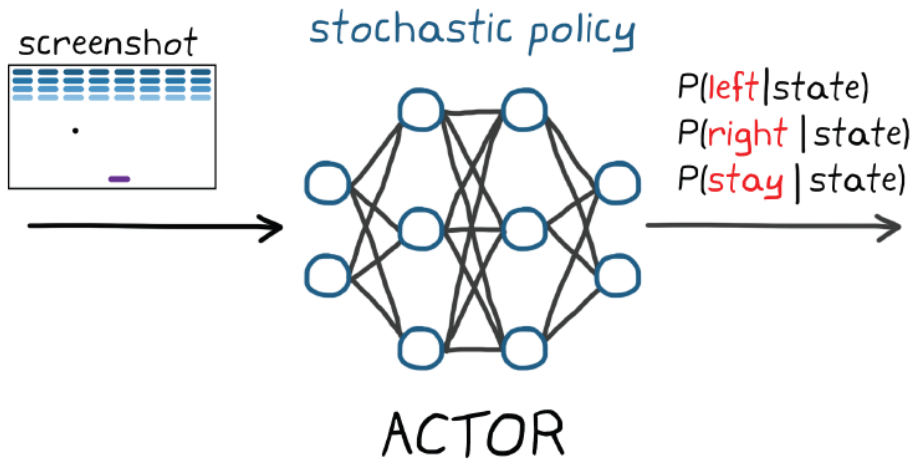
Policy란?



$$\text{policy} \\ \text{actions} = \text{function}(\text{state observations})$$

Function을 무엇으로 구현할까?

Policy를 신경망으로 구현



강화학습 구현을 위한 순서



1. Environment 구축

- Action에 따라 서로 다른 Observation을 얻을 수 있는 Environment 구축 (테스트베드 or Simulation)

2. Reward 설계

- 사람이 직접 알고리즘을 설계 (이론적/경험적 지식에 따라)

3. Policy 설계

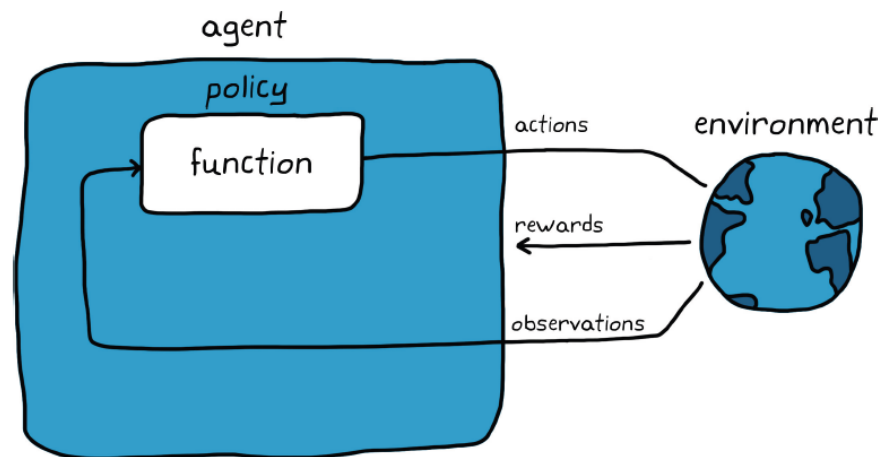
- 어떤 Policy 모델과 옵션을 사용할 것인지

4. Training

- Agent 훈련

5. Deploy

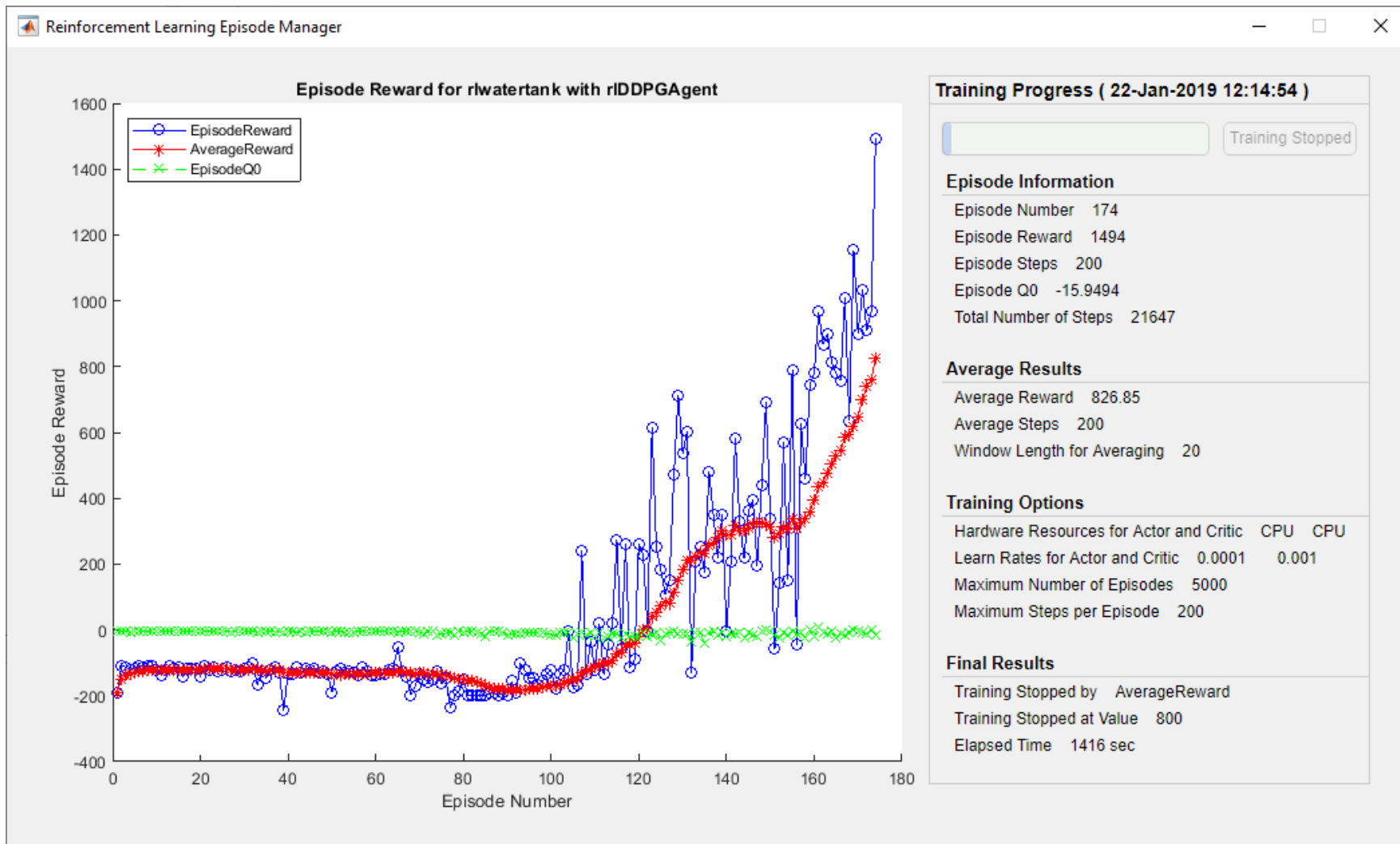
- 훈련이 완료된 Agent를 실제 시스템에 적용



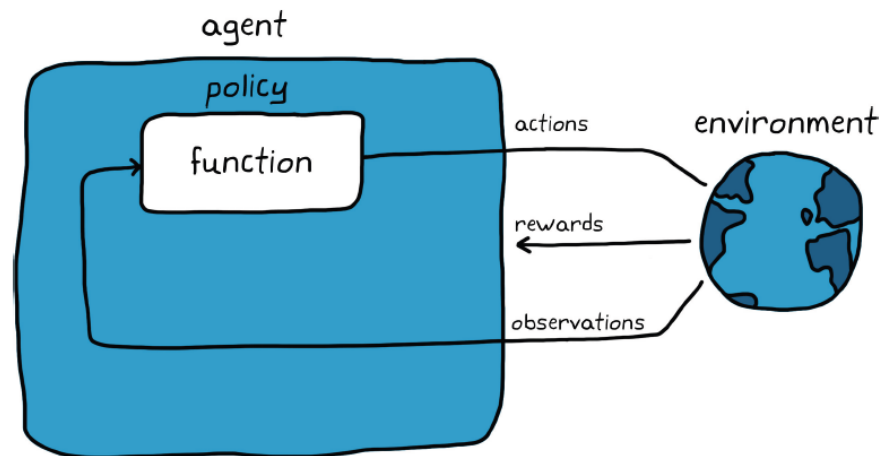
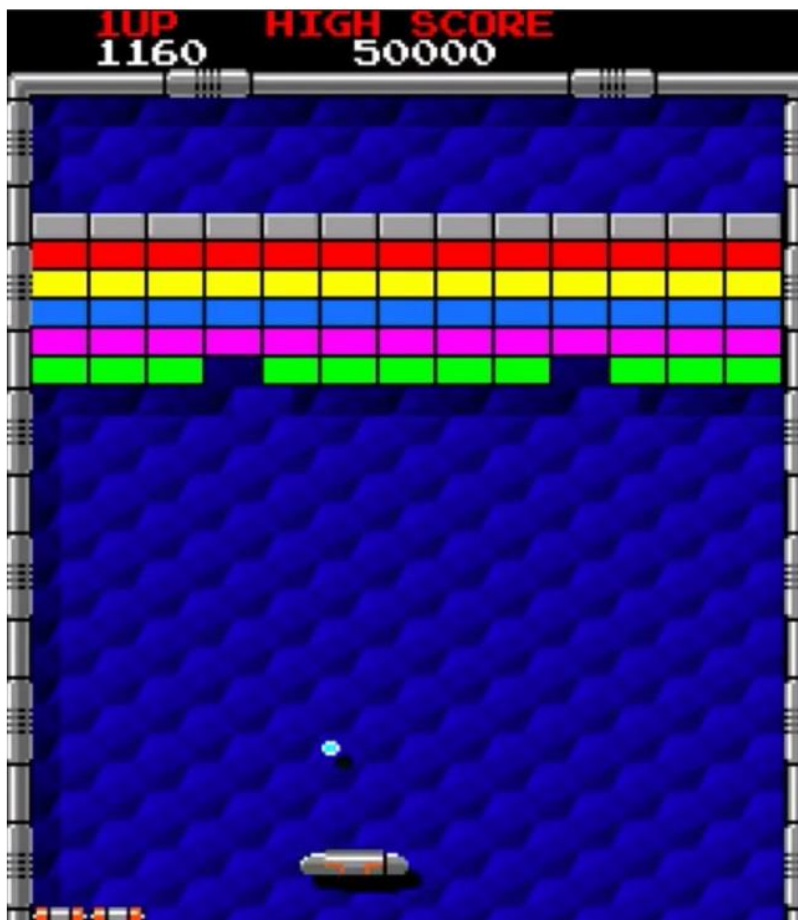
강화학습 Training 결과 예시



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.



생각해봅시다



1. Action을 어떻게 설정하면 좋을까?
2. Observation을 어떻게 설정하면 좋을까?
3. Environment는 어떻게 구축 해야할까?
4. Reward를 어떻게 설계하면 좋을까?

1. 머신러닝 + 딥러닝 개념을 이해한 상태여야만 입문할 수 있다.
2. 도메인 적용을 위해서는 도메인에 대한 전문적인 지식이 요구된다.



강화학습 && 도메인 지식 => 강화학습 응용 가능

Ex) 알파고 제로 (강화학습 기반) 개발 당시,
프로 기사 출신의 박사가 개발팀에 있었음.

1. 도메인 전문가가 강화학습을 공부하거나,
2. 강화학습 전문가가 도메인을 공부하거나

- Keras의 MNIST 데이터 set 불러오기

```
from tensorflow import keras

mnist = keras.datasets.mnist
(X_train_full, Y_train_full), (X_test, Y_test) = mnist.load_data()
```

- Train set, Validation set 으로 나누기 (총 5000개의 데이터만 사용)

```
X_valid, X_train = X_train_full[:500], X_train_full[500:5000]
Y_valid, Y_train = Y_train_full[:500], Y_train_full[500:5000]
```

- models.Sequential을 이용하여 CNN 모델 생성

```
model = keras.models.Sequential([
    keras.layers.Conv2D(128, (3,3), activation='relu',
                        strides=(1,1), input_shape=(28,28,1)),
    keras.layers.MaxPool2D((2,2)),
    keras.layers.Flatten(),
    keras.layers.Dense(units=64, activation='relu'),
    keras.layers.Dense(units=10, activation='softmax')
])
```

- 모델 요약 및 학습 방식 환경 설정

```
model.summary()

model.compile(optimizer = 'adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

- 학습 진행

```
history = model.fit(X_train, Y_train, epochs=10, batch_size=64,
                    validation_data=(X_valid, Y_valid))
```

- Test set을 이용하여 정확도 확인

```
model.evaluate(X_test, Y_test)
```



```
313/313 [=====] - 0s 1ms/step - loss: 0.1374 - accuracy: 0.9613
Out[70]: [0.1374218463897705, 0.9613000154495239]
```


- 예측 결과 확인

- 학습 결과를 그래프로 확인

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

pd.DataFrame(history.history).plot(figsize=(8,5))
plt.grid(True)
plt.gca().set_ylim(0,1)
plt.show()
```

각 실습문제에 대해 *code(+주석)*, *그래프*, *분석 내용* 등은 필수적으로 포함시킬 것

■ Keras를 활용한 CNN 구현

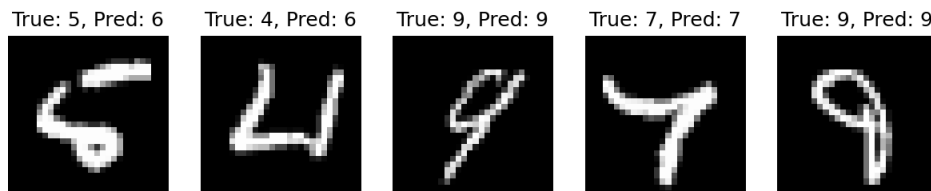
- MNIST 데이터 set 불러온 후, Trainig set : Test set 분할
- Training set을 Training set, Validation set 으로 분할
(조건 : Train + Validation 개수 5000개로 제한)
- 모델 구조를 수정하여 자신만의 CNN 모델 만들기
- 학습 후, Test set을 이용하여 정확도 확인

Pass 기준 :

⇒ Epoch에 따른 Accuracy 및 MSE 그래프 그리기(*train , Validation set에 대해*)

⇒ 아래 그림과 같이, 잘못 예측된 이미지 시각화

[잘못 예측된 이미지 시각화 예시]



[Hint code] 잘못 예측한 값



- 잘못 예측한 값과 실제 값 비교 후 이미지 시각화

```
pred = model.predict(X_test) # test data 예측
pred_class = np.argmax(pred, axis=1)
true_class = np.argmax(Y_test, axis=1)

mis_index = np.where(pred_class != true_class) # 잘못 예측된 인덱스 찾기
mis_index = np.array(mis_index).reshape(-1)

# 잘못 예측된 이미지 시각화 함수
plt.figure(figsize=(10, 10), dpi=150) # 그림 크기 설정
for i in range(5): # 첫 5개의 잘못 예측된 이미지만 표시
    index = mis_index[i]
    plt.subplot(1, 5, i + 1) # 1행 5열의 서브플롯 생성
    plt.axis('off')
    plt.grid(False) # 그리드 제거
    plt.imshow(X_test[index, :, :], cmap='gray') # 이미지 표시 (흑백으로)
    plt.title(f'True: {true_class[index]}, Pred: {pred_class[index]}')
plt.show()
```